Container/Presentational Pattern

2 min read Updated 24 Mar 2026 Topic 30 of 62

Bookmark Mark done

6.1 – Container/Presentational Pattern

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is the Container/Presentational pattern?"	🟡	★★★★☆
"What is the difference between smart and dumb components?"	🟡	★★★☆☆
"Is this pattern still relevant with hooks?"	🟡🔴	★★★☆☆

The Pattern

The idea is simple: separate **what data to show** from **how to show it**.

Container components (smart) handle data fetching, state management, and business logic. They don't render any UI of their own. They pass data down as props.

Presentational components (dumb) receive data via props and just render it. They have no idea where the data comes from. They're pure visual components.

JSX

Copy

```jsx
// Container: handles data, no UI
function UserListContainer() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => { setUsers(data); setLoading(false); });
  }, []);

  if (loading) return <Spinner />;
  return <UserList users={users} />;
}

// Presentational: renders UI, no logic
function UserList({ users }) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>
          <img src={user.avatar} alt={user.name} />
          <span>{user.name}</span>
        </li>
      ))}
    </ul>
  );
}
```

Is It Still Relevant?

With hooks, you can put data fetching logic directly in a component or extract it into a custom hook (like `useFetch`). This blurs the line between containers and presentational components.

separate components or through custom hooks, the goal is the same.

Interview answer: "The Container/Presentational pattern separates data logic from UI rendering. Containers fetch data and manage state, presentational components just display what they receive via props. With hooks, I often achieve the same separation by extracting logic into custom hooks instead of separate container components. The underlying principle of separating concerns is still the best practice."

QUICK CHECK — no marks, just for you.

1. According to the lesson, what is the responsibility of a container (smart) component?

- ○ It handles data fetching, state management, and business logic, and passes data down as props without rendering UI of its own
- ○ It memoizes expensive renders so presentational children never re-render unnecessarily
- ○ It defines the prop types and default props that presentational components will validate against
- ○ It renders the visual markup and delegates only the styling decisions to child components

Check answer

2. The lesson describes presentational (dumb) components. Which statement matches that description?

- ○ They receive data via props and just render it, with no knowledge of where the data came from
- ○ They subscribe directly to the global store but never dispatch actions to it
- ○ They must be written as class components so they can hold local UI state

They fetch their own data but delegate loading and error states to a parent container

Check answer

3. The lesson asks whether this pattern is still relevant now that hooks exist. What conclusion does it reach?

- ○ Hooks replace presentational components, so all rendering should move into the container
- ○ The pattern is obsolete because hooks make any separation of data and UI concerns unnecessary
- ○ The strict split into separate components matters less, but separating data concerns from visual concerns is still essential and can be achieved with custom hooks
- ○ The pattern is now mandatory, since custom hooks can only be called from container components

Check answer

‹ Previous
Web Vitals, Tree Shaking & Bundle Optimization

Next ›
Higher-Order Components (HOC)

LEARN MORE

ARTICLES & DOCS

Thinking in React (split UI)
react.dev

VIDEOS

React.js Interview Kit



Container & presentational — beginner friendly

YouTube



Container/presentational — step by step

YouTube

Hand-picked references for this lesson.